Lab 10: State-Based Control Flow Using RISC-V Assembly

Objective

The objective of this lab is to implement a Finite State Machine (FSM) entirely in assembly language using switches and LEDs as memory-mapped I/O devices. Students will apply previously learned concepts of address decoding while focusing on control flow design, stack usage, and instruction execution.

This lab reinforces software–hardware interaction by requiring students to:

- Implement FSM behavior using assembly-level control flow
- Use conditional branching to represent FSM state transitions
- Employ the stack to support function calls and register preservation
- Store and execute assembly instructions from an instruction memory module

The functional behavior of the FSM remains identical to that implemented in Lab 5.

Processor Clock Frequency and Real-Time Behavior

Processor Clock Rate

In this lab, the RISC-V processor operates at a fixed clock frequency of:

$$f_{clk} = 10 \text{ MHz}$$

This means the processor executes:

$$10,000,000 \text{ clock cycles per second}$$

Every instruction, memory access, branch, and loop iteration consumes one or more clock cycles. Therefore, software execution time directly depends on the clock frequency.

Why This Matters

This directly affects:

- Delay loops
- Software timers
- Counter-based delays
- Busy-wait routines

For example, a loop that runs 100 million iterations will take:

$$\frac{100,000,000}{10,000,000} = 10 \text{ seconds}$$

This means poorly designed delay loops can easily cause unexpected long delays or register overflows.

Design Implication for This Lab

Since the FSM countdown and polling loops are implemented in assembly, students must account for clock-dependent execution time to ensure responsive and correct behavior.

System Description

The system behavior is defined as follows:

- Switches, LEDs, and the reset button are accessed as memory-mapped locations.
- The system continuously reads the switch input value.
- If the switch input value is **zero**, the system remains in the input-waiting state.
- When the switch input value becomes **non-zero**:
  - The value is captured
  - The value is displayed on the LEDs
  - A decrement counter begins, counting from the captured value down to zero
- During the countdown:
  - The LEDs display the current counter value
  - Further switch input is ignored
- When the counter reaches zero:
  - The FSM automatically returns to the input-waiting state
- Pressing the reset button at any time during the countdown:
  - Immediately resets the FSM to the input-waiting state
  - Clears the counter and LED output

Note: Students are expected to use memory-mapped I/O knowledge acquired in the address decoding lab.

Task 1: FSM Design in Assembly

Objective

To translate the FSM logic into an assembly-based control flow using labels, branches, and state variables.

Procedure

(a) Review the FSM designed in Lab 5.

(b) Identify the required states, such as:

- Input Waiting State

- Countdown State

(c) Represent FSM states using:

- State variables stored in registers or memory
- Assembly labels and conditional branches

(d) Draw a control flow diagram showing:

- State transitions
- Branch conditions
- Reset behavior

Deliverables

- FSM control flow diagram for the assembly implementation

Task 2: Assembly Implementation of the FSM

Objective

To implement the complete FSM behavior in assembly language using control flow, stack-based function calls, and memory-mapped I/O.

Procedure

(a) Write an assembly program that:

- Continuously reads switch input values
- Implements FSM state transitions using conditional branches
- Displays values on LEDs according to the current FSM state

(b) Implement the countdown logic as a subroutine:

- Pass the initial count value as an argument
- Decrement the value until zero
- Update LED output at each step

(c) Use the stack to:

- Save and restore registers
- Preserve return addresses during subroutine calls

(d) Ensure reset detection immediately returns execution to the input-waiting state.

Deliverables

- Complete assembly source file implementing the FSM
- Demonstration of correct stack usage in the code

Task 3: Instruction Memory Module Design

Objective

To design and implement an instruction memory module that stores and supplies the FSM assembly instructions to the processor.

Procedure

(a) Design an instruction memory module in Verilog:

- Store the assembled machine code instructions
- Output instructions based on the program counter (PC)

(b) Initialize the instruction memory using the reference provided:

```
module instructionMemory#(
    parameter OPERAND_LENGTH = 31
    )(
    input [OPERAND_LENGTH:0] instAddress,
    output reg [31:0] instruction
);
reg [7:0] memory [0:255];
...
endmodule
```

(c) Populate the instruction memory with the machine language translation of the assembly code of Task 2.

(d) Verify correct instruction sequencing during execution.

Deliverables

- Verilog instruction memory module
- Memory initialization file containing assembly instructions

Assessment Rubric
Lab 10: State-Based Control Flow Using RISC-V Assembly

Name:	Student ID:	Section:

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/20	-
	Task 3	/20	-	-
Total Points		**/40**	**/20**	**/20**
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	**Report Submission & Git Upload**	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	**100**	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.